SYSTEM DESIGN & SOFTWARE ENGINEERING

PROJECT REPORT

HostelHub

Smart Hostel Management System

A full-stack web application designed to modernize hostel administration through scalable, modular architecture

PROJECT DETAILS	
Course	SDSE — Academic Project
Team Size	5 Developers
Date	April 27, 2026
Status	Completed

Nitin Kumar • Manjeet • Prachee • Mayank Yadav • Arun

1. Executive Summary

HostelHub is a comprehensive, full-stack web application developed as part of the System Design and Software Engineering (SDSE) academic course. The platform is engineered to modernize hostel administration by automating key operational workflows including room allocation, complaint tracking, leave management, and student record maintenance.

Project Name	HostelHub: Smart Hostel Management System
Objective	Automate hostel operations with scalable, modular architecture
Course	System Design & Software Engineering (SDSE)
Team Size	5 Developers
Key Deliverables	Full-stack web app with role-based access, real-time management, and reporting

The system delivers role-based access control for three user types (Student, Admin, Staff), real-time complaint and leave management, automated room allocation workflows, and a rich analytics dashboard — all implemented using modern web technologies and advanced software engineering principles.

2. Problem Statement & Proposed Solution

2.1 Problem Analysis

Traditional hostel management relies heavily on manual processes that introduce significant administrative overhead, prone to human error and inefficiency. Key challenges identified include:

- Manual room allocation is time-consuming and prone to double-booking errors
- Wardens lack a unified platform for managing complaints, leave requests, and student records
- Students have no transparent mechanism to track the status of their requests
- Small and mid-size hostels lack access to affordable, integrated digital solutions
- Generating occupancy and compliance reports is a laborious manual process

2.2 Proposed Solution

HostelHub provides a centralized digital platform that addresses each challenge through purpose-built modules:

- Automated Room Allocation: Real-time availability checks with transaction-backed assignments
- Complaint Tracking: Lifecycle management from submission through resolution
- Leave Request Management: Structured approval workflow with student notifications
- Student Record Maintenance: Centralized, searchable profile and enrollment data
- Analytics Dashboard: Occupancy rates, complaint trends, and leave summaries at a glance

3. System Architecture & Design

3.1 Architectural Approach — Layered Architecture

HostelHub implements a classic four-layer architecture that enforces separation of concerns, promotes testability, and supports independent scaling of each tier.

Layer	Technology	Responsibility
Presentation	React.js + TailwindCSS	Component-driven UI, routing, state management
Controller	Express.js Controllers	HTTP request handling and response formatting
Service	TypeScript Service Classes	Business logic and domain operations
Repository	Prisma ORM + MySQL	Data persistence and query management

3.2 Technology Stack

Component	Technology	Version
Frontend	React.js	19.2.4
Frontend Build	Vite	8.0.1
Backend	Node.js + Express	5.2.1
Database	MySQL (via Prisma)	—
ORM	Prisma	6.8.2
Authentication	JWT + bcryptjs	—
Validation	Zod	4.3.6
Styling	TailwindCSS + PostCSS	4.2.2
HTTP Client	Axios	1.14.0
Routing	React Router DOM	7.13.2
Data Visualization	Recharts	3.8.1
Type System	TypeScript	5.9.3+

3.3 Design Patterns Implemented

Five established design patterns were applied to solve recurring architectural challenges, improving modularity, extensibility, and code clarity.

Singleton Pattern	
Application	Database Connection Manager (DatabaseManager)
Purpose	Ensures a single shared instance of the Prisma client is maintained across all service calls, preventing resource exhaustion.
Code Reference	server/src/interfaces/IServices.ts

Service Registry Pattern	
Application	Centralized service management (ServiceRegistry)
Purpose	Provides a single, consistent access point for all service instances, decoupling controllers from direct service instantiation.
Code Reference	server/src/interfaces/ServiceRegistry.ts

Factory Pattern	
Application	Role-based user object creation
Purpose	Creates the appropriate user class instance (Student, Admin, or Staff) at signup and login time based on the user's role.
Code Reference	server/src/services/AuthService.ts

Strategy Pattern	
Application	Pluggable room allocation logic
Purpose	Encapsulates different room allocation algorithms as interchangeable strategies, enabling flexible assignment logic without modifying existing service code.
Code Reference	server/src/services/RoomService.ts

Observer (Planned) Pattern	
Application	Notification broadcasting system
Purpose	Architecture is designed to support event-driven notifications for room allocation confirmations and fee payment reminders.
Code Reference	Planned — Notification module

3.4 SOLID Principles Analysis

Principle	Implementation	Example
SRP	Each service handles exactly one domain	StudentService, RoomService, ComplaintService are fully independent
OCP	Extension without modification	New room types and payment methods added via enum without code changes
LSP	Safe base class substitution	Student, Admin, Staff safely substitute the base User class
ISP	Focused, minimal interfaces	IUserService, IRoomService — no fat interface anti-patterns
DIP	Depend on abstractions	Controllers depend on ServiceRegistry, not on concrete service classes

4. Database Design

4.1 Core Entities

The database schema is organized around three user entities and five domain entities, with carefully designed relationships to maintain data integrity.

User Hierarchy

- User — Base entity: email, hashed password, role enum (STUDENT, ADMIN, STAFF)
- Student — Extends User: enrollment number, course, year, contact, parent info
- Admin — Extends User: administrative privileges and system access
- Staff — Extends User: maintenance/cleaning role-specific permissions

Domain Entities

- Room — Capacity, type, amenities, fee structure, current status
- RoomAllocation — Student-to-room assignments with ACTIVE/VACATED/TRANSFERRED status
- Complaint — Student-submitted issues with full lifecycle status tracking
- LeaveRequest — Leave applications with date ranges and approval workflow
- CleaningRequest — Room maintenance service requests
- Notification — System alerts and reminders for all user types

4.2 Key Enumerations

Enum	Values
Role	STUDENT | STAFF | ADMIN
RoomType	SINGLE | DOUBLE | TRIPLE | DORMITORY
RoomStatus	AVAILABLE | OCCUPIED | MAINTENANCE | RESERVED
AllocationStatus	ACTIVE | VACATED | TRANSFERRED
FeeStatus	PENDING | PAID | OVERDUE | WAIVED
PaymentMethod	ONLINE | CASH | CHEQUE | BANK_TRANSFER

4.3 Database Optimizations

- Composite indexes on student_id and room_id for high-frequency join queries
- Pagination on all list endpoints with configurable page size (default: 10 records)

- Atomic transactions for critical multi-step operations (room allocation, fee payments)
- Strategic denormalization of frequently accessed room data in allocation records
- Prisma connection pooling for efficient database resource management

5. OOP Concepts Demonstration

5.1 Encapsulation

All domain entities encapsulate internal state using private fields and expose controlled access via getter and setter methods. Setters include validation logic — for example, student year is constrained to the range 1–6, preventing invalid state from ever being stored.

- Private fields: _enrollmentNumber, _roomNumber, _password
- Controlled access via public getters and validated setters
- Reference: server/src/classes/Student.ts

5.2 Inheritance

A base User class provides shared infrastructure — authentication utilities, profile management, and permissions scaffolding. The derived classes Student, Admin, and Staff inherit this foundation and extend it with role-specific behavior.

- Base class: server/src/classes/User.ts — shared auth, profile, permissions
- Derived classes: Student.ts, Admin.ts, Staff.ts — specialized attributes
- Code reuse: authentication, token validation, profile formatting

5.3 Polymorphism

The getPermissions() and getProfileSummary() methods are overridden in each subclass, providing role-appropriate implementations while sharing a common calling interface:

- Student permissions: view_room_details, submit_complaint, apply_leave
- Admin permissions: manage_users, generate_reports, override_allocations
- Staff permissions: handle_complaints, manage_cleaning, view_assignments

5.4 Abstraction

Controllers interact exclusively with service abstractions and never directly with Prisma or database constructs. The DatabaseManager class hides all connection management complexity behind a clean interface, allowing business logic to remain portable and testable.

6. Backend Implementation

6.1 Core Services

AuthService

- Student signup with transactional user + student record creation
- Credential-based login with bcryptjs hash comparison (10 rounds)
- JWT token generation with configurable expiration (default: 7 days)
- Role extraction embedded in token payload for stateless authorization

RoomService

- allocate(): Validates eligibility and availability, executes atomic transaction
- deallocate(): Removes assignment, resets room status in single transaction
- getAvailable(): Filtered query returning rooms with remaining capacity
- getStudentRoom(): Retrieves current active allocation for a given student

StudentService, ComplaintService, LeaveService

- Domain-specific CRUD and workflow operations
- Pagination support on all list operations
- Multi-step workflows protected by database transactions

6.2 Authentication & Authorization

- Stateless JWT authentication — no server-side session storage
- authMiddleware.ts validates token signature and expiration on every request
- Role field extracted from token and used for per-route authorization
- Unauthorized requests return HTTP 401; forbidden roles return HTTP 403

6.3 Input Validation

- All API inputs validated with Zod schemas before entering service layer
- Strongly typed validation with detailed, field-level error reporting
- authValidators.ts and domainValidators.ts cover all endpoints
- Schema validation errors are serialized and returned with HTTP 400

7. Frontend Implementation

7.1 State Management

- AuthContext: Manages user session, token persistence, and role-based routing
- ToastContext: Centralized toast notification queue for user feedback
- Custom hooks (useAuth, useDashboard, useRoomData, useComplaints): Domain state encapsulation
- Component-level state: Forms, UI toggles, loading indicators

7.2 Component Architecture

- Page Components: Full-page views (Login, Dashboard, Complaints, Leave, Profile)
- Layout Components: Reusable structure (Header, Navigation, Footer)
- UI Components: Atomic, reusable elements (Buttons, Forms, Tables, Modals)
- ProtectedRoute.tsx: Guards all pages requiring authentication and role verification

7.3 Service Layer

- authService.ts, roomService.ts, complaintService.ts etc. encapsulate Axios API calls
- JWT token automatically injected into every authenticated request header
- Consistent error handling pattern across all API communication
- TypeScript interfaces in types/ folder enforce request and response shapes

7.4 UI Features

- Responsive Design: TailwindCSS mobile-first layout adapts to all screen sizes
- Data Visualization: Recharts library renders occupancy, complaint, and leave analytics
- Toast Notifications: Real-time user feedback for all async operations
- Lazy Loading: Page-level code splitting via Vite for optimized bundle sizes

8. Modular Features

8.1 Student Module

The student-facing interface provides a self-service portal with full visibility into room assignments, complaint status, leave applications, and fee records.

- Profile management: course, year, contact, parent information
- Room viewing and real-time allocation status
- Complaint submission with description and category
- Leave request application with date range and reason
- Fee records viewing with payment history
- Support ticket creation for administrative assistance

8.2 Admin Module

Administrators have full system oversight with tools for student management, room operations, and comprehensive reporting.

- Student management: add, edit, and remove student records
- Room allocation and manual assignment override capability
- Report generation: occupancy, complaint, and leave summaries
- Dashboard analytics with Recharts visualizations

8.3 Staff Module

- Complaint handling: accept, update status, and resolve assigned complaints
- Cleaning task management and scheduling
- Room maintenance status tracking and assignment viewing

9. Key System Workflows

9.1 Room Allocation Workflow

1. Admin views available rooms with real-time capacity status
2. System validates student eligibility (not already allocated)
3. Room capacity constraints verified against type (SINGLE/DOUBLE/TRIPLE/DORMITORY)
4. Student-Room allocation created inside a database transaction
5. Room status updated atomically: AVAILABLE → OCCUPIED
6. Allocation record stored with ACTIVE status and timestamp
7. Confirmation notification generated for the student

9.2 Complaint Management Workflow

8. Student submits complaint with description and category
9. Complaint recorded with PENDING status and timestamp
10. Admin and relevant Staff notified of new complaint
11. Staff member assigned to the complaint
12. Status progresses: PENDING → IN_PROGRESS → RESOLVED
13. Student receives notifications at each status change
14. Complaint archived with resolution notes on closure

9.3 Leave Request Workflow

15. Student submits leave application with start/end dates and reason
16. Request stored with PENDING approval status
17. Admin reviews and evaluates the application
18. Admin approves or rejects with comments
19. Student notified of decision via notification system
20. Approved leave updates room allocation status accordingly

9.4 Authentication Workflow

21. User submits email and password via login page
22. Zod schema validates input format before processing
23. Credentials looked up in database; password compared with bcrypt hash
24. JWT token generated on successful authentication
25. Token stored client-side and included in all subsequent API request headers

26. authMiddleware validates token signature and expiration on each request
27. Requests with invalid or expired tokens rejected with HTTP 401

10. Security Measures

10.1 Authentication Security

- Password hashing: bcryptjs with 10 rounds of salting — rainbow table resistant
- JWT tokens signed with server-side secret, include expiration timestamp
- Token stored in localStorage on client; transmitted via Authorization header
- Application configured for HTTPS-ready deployment

10.2 Authorization Security

- Role-Based Access Control: STUDENT, ADMIN, STAFF with distinct permission sets
- ProtectedRoute.tsx guards all frontend routes requiring authentication
- Backend middleware validates role claims per route before any service call
- Scope isolation: users can only query and modify their own data

10.3 Data Protection

- All user inputs validated through Zod schemas before processing
- Prisma parameterized queries prevent SQL injection attacks
- Express CORS middleware restricts cross-origin access to trusted origins
- Environment variables separate sensitive configuration from source code

11. Testing Strategy & Results

11.1 Authentication Module

- ✓ Student signup with valid credentials
- ✓ Student signup with duplicate email (correctly rejected)
- ✓ Student login with correct credentials
- ✓ Student login with incorrect password (correctly rejected)
- ✓ Admin and Staff login functionality
- ✓ JWT token generation and validation

11.2 Room Management

- ✓ View available rooms with capacity status
- ✓ Allocate student to available room
- ✓ Prevent allocation to full room (capacity exceeded)
- ✓ Deallocate student from room
- ✓ Room status updates correctly on allocation/deallocation
- ✓ Prevent deletion of room with active allocations

11.3 Complaint & Leave Management

- ✓ Student submit complaint
- ✓ Admin view and filter all complaints
- ✓ Update complaint status through lifecycle
- ✓ Assign complaint to staff member
- ✓ Student apply for leave with date range
- ✓ Admin approve and reject leave with comments
- ✓ Generate complaint and leave reports

11.4 Edge Cases & Validation

- ✗ Invalid email format in signup
- ✗ Password under 8 characters
- ✗ Negative room capacity input
- ✗ Student year outside valid range (1–6)

- ✗ Leave end date before start date
- ✗ Duplicate enrollment number in system
- ✗ Unauthorized access attempt to protected route

11.5 Concurrency Testing

- ✓ Two simultaneous room allocations — only one succeeds
- ✓ Concurrent complaint status updates from multiple staff
- ✓ Database transaction rollback on allocation failure

12. Scalability & Performance

12.1 Scalability Measures

- Stateless REST APIs enable horizontal scaling without session affinity
- Database indexes on student_id and room_id for constant-time lookups
- All list endpoints paginated (default 10 items/page) to bound response sizes
- Modular service design supports future microservice decomposition
- Prisma connection pooling manages database connections efficiently

12.2 Performance Optimizations

- Selective Prisma field retrieval avoids over-fetching unnecessary columns
- Batch processing for report generation on large datasets
- In-memory caching for frequently accessed room availability data
- Vite build with tree-shaking and code splitting for optimized frontend bundles
- Lazy-loaded page components reduce initial JavaScript bundle size

13. Future Enhancements

13.1 Planned Feature Additions

28. Payment Integration — Online payment gateway for fee collection (Razorpay/Stripe)
29. Email Notifications — Automated alerts for allocations, approvals, and reminders
30. SMS Alerts — SMS notifications for critical, time-sensitive events
31. Advanced Analytics — Predictive occupancy modeling and trend analysis
32. Mobile App — Native iOS and Android applications
33. Real-time Updates — WebSocket support for live notification delivery
34. Document Upload — Complaint photos, fee receipts, and ID document storage
35. Maintenance Scheduling — Automated recurring room maintenance calendar
36. Guest Management — Visitor tracking and approval workflow
37. Audit Logging — Complete, tamper-evident trail for compliance reporting

13.2 Technical Improvements

38. Rate Limiting — Protect API endpoints from abuse and denial-of-service
39. Redis Caching — Session and data caching layer for improved response times
40. OpenAPI/Swagger — Comprehensive API documentation for integration partners
41. Jest/Mocha Unit Tests — Automated test suites with coverage reporting
42. End-to-End Testing — Playwright or Cypress automated integration tests
43. Load Testing — k6 or JMeter benchmarking for performance validation
44. Database Replication — Master-slave setup for high availability
45. Centralized Log Aggregation — ELK stack or similar for operational monitoring

14. Project Statistics & Team

14.1 Codebase Metrics

Metric	Value
React Components	20+ components
Backend Services	9+ service classes
API Routes	5+ modules, 30+ endpoints
Database Entities	8+ core entities
Backend Lines of Code	~5,000+ lines
Frontend Lines of Code	~3,000+ lines

14.2 Development Team

Member	Student ID	Role
Nitin Kumar	2401010305	Team Lead and Full stack Developer
Manjeet	2401010262	Frontend Developer
Prachee Dhar	2401010330	Frontend Developer
Mayank Yadav	2401010271	QA & Testing
Arun	2401010098	Backend Developer

15. Conclusion

HostelHub successfully demonstrates the application of advanced software engineering principles to a practical, real-world problem domain. The project delivers a fully functional hostel management platform while serving as a showcase of modern full-stack development practices.

15.1 Learning Objectives Achieved

Objective	Implementation
Layered Architecture	Controller → Service → Repository → DB fully implemented
OOP Concepts	Encapsulation, Inheritance, Polymorphism, Abstraction demonstrated
Design Patterns	Singleton, Registry, Factory, Strategy, Observer (planned)
SOLID Principles	All five principles applied across the codebase
REST API Design	30+ endpoints following REST conventions
Database Design	Normalized schema with indexing and transaction management
Security	JWT auth, bcrypt hashing, Zod validation, CORS protection
Team Collaboration	5-member team delivering full-stack application

HostelHub represents more than an academic exercise — it is a production-ready foundation for a real hostel management product. With a clean layered architecture, comprehensive role-based access control, and a modular codebase designed for extension, the platform is positioned for the planned enhancements outlined in Section 13. The team's disciplined application of SOLID principles and established design patterns ensures the system will remain maintainable as it grows.